

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I **KESHAVARAM L/192511253** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

L. Keshavaram

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables. Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions. Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers. Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains. Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices. Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice

Case Study 1: Weak Password Hashing in Web Application

Scenario

A web application stores user passwords using unsalted SHA-1 hashes. After a database breach, attackers use precomputed tables to recover many passwords quickly.

Analysis

The major weakness is the use of SHA-1 as a password-hashing algorithm without a salt. SHA-1 is designed to be computationally fast, which is undesirable for password storage because attackers can test huge numbers of password guesses quickly.

The absence of a unique random salt means that users with the same password produce the same hash. Attackers can therefore use precomputed rainbow tables to identify common passwords without calculating every possibility from scratch.

Security Impact

- Passwords can be cracked rapidly.
- Identical passwords produce identical hashes.
- Rainbow-table attacks become practical.
- Compromised passwords may be reused on other websites.
- SHA-1 is no longer considered suitable for modern security applications.

Recommended Solution

Use a password-specific hashing algorithm such as Argon2id, bcrypt, or PBKDF2, with a unique random salt for every password.

Justification

A secure password-hashing algorithm deliberately makes password guessing expensive. A unique salt ensures that identical passwords have different stored values, while key stretching increases the computational cost of each password guess.

Conclusion

The combination of unsalted SHA-1 and fast hashing caused the vulnerability. Replacing it with salted Argon2id/bcrypt/PBKDF2 significantly improves resistance to brute-force and precomputed-table attacks.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents

Case Study 2: Digital Signature Compromise in Financial System

Scenario

A financial institution uses digital signatures to approve transactions. An attacker obtains a user's private signing key and uses it to authorize fraudulent transactions.

Analysis

A digital signature depends on the secrecy of the private key. The private key is used to generate signatures, while the corresponding public key is used for verification.

Once an attacker obtains the legitimate private key, the attacker can generate mathematically valid signatures. The system may therefore interpret fraudulent transactions as being authorized by the legitimate key owner.

Security Impact

Authentication:

The attacker can impersonate the legitimate signer because valid signatures can be produced using the stolen private key.

Integrity:

The attacker can create unauthorized transactions that appear to have been approved.

Non-repudiation:

Non-repudiation is weakened because the private key was compromised. A valid signature alone cannot prove that the legitimate user personally created the transaction if the key was accessible to an attacker.

Recommended Solutions

1. Store private keys in Hardware Security Modules (HSMs) or secure hardware.
2. Use multi-factor authentication before transaction approval.
3. Implement transaction limits and risk-based verification.

4. Maintain certificate/key revocation mechanisms.
5. Rotate compromised keys immediately.
6. Monitor unusual transaction activity.
7. Use secure key backup and recovery procedures.

Justification

Keeping private keys inside protected hardware makes unauthorized extraction much more difficult. Key revocation prevents a compromised key from continuing to be trusted.

Conclusion

A compromised private key directly threatens authentication, integrity, and non-repudiation. Strong key protection, monitoring, MFA, and rapid key revocation are essential in financial systems.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

Case Study 3: Kerberos Authentication Failure in Distributed System

Scenario

A distributed system uses Kerberos, but users frequently experience authentication failures because the clocks of different servers are not synchronized.

Analysis

Kerberos uses timestamps in authentication tickets to prevent replay attacks. A client, authentication server, ticket-granting server, and application server must therefore maintain reasonably synchronized clocks.

For example:

Client Time → 10:00:00

Kerberos Server → 10:08:30

Application Server → 10:08:35

If the difference exceeds the configured clock-skew tolerance, the server may reject the authentication ticket.

Security Impact

- Legitimate users may be denied access.
- Authentication becomes unreliable.
- Applications depending on Kerberos may stop functioning.
- Incorrect time handling can weaken replay protection.
- Distributed services may experience cascading authentication failures.

Recommended Solutions

1. Deploy NTP (Network Time Protocol).
2. Use reliable internal time servers.
3. Configure appropriate clock-skew tolerance.
4. Monitor clock synchronization continuously.
5. Protect NTP infrastructure from manipulation.
6. Synchronize all domain controllers and application servers.

Justification

NTP maintains consistent time across distributed systems. Proper synchronization allows Kerberos to validate timestamps correctly while preserving its protection against replay attacks.

Conclusion

Time synchronization is a critical requirement for Kerberos. Reliable and secure NTP infrastructure ensures both authentication availability and replay protection.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

Case Study 4: Certificate Authority Breach in PKI System

Scenario

A Certificate Authority is compromised and attackers issue fake X.509 certificates for trusted domains.

Analysis

A Certificate Authority is a trust anchor in a Public Key Infrastructure (PKI). When a CA issues a certificate, browsers and other systems may trust the certificate because they already trust the CA.

The attack can therefore create a false relationship:

Compromised CA



Fake Certificate



Trusted Domain



Attacker impersonation

An attacker could potentially use a fraudulent certificate to impersonate a website or intercept encrypted communications.

Security Impact

- Trust in the PKI is compromised.
- Users may unknowingly connect to an attacker.
- Man-in-the-middle attacks become possible.
- Confidential communications may be exposed.
- The reputation of the CA and affected organizations may be damaged.

Recommended Solutions

1. Revoke fraudulent certificates.
2. Use Certificate Revocation Lists (CRLs).
3. Use OCSP to check certificate status.
4. Implement Certificate Transparency (CT) monitoring.
5. Revoke or distrust compromised CA certificates where necessary.
6. Replace compromised keys and certificates.
7. Use strong CA security controls and HSMs.
8. Monitor unexpected certificate issuance.

Justification

Certificate Transparency provides publicly auditable records of certificate issuance, making unauthorized certificates easier to detect. Revocation mechanisms allow systems to stop trusting compromised certificates.

Conclusion

A CA compromise threatens the entire trust chain of PKI. Certificate Transparency, revocation, monitoring, secure CA key storage, and rapid incident response are essential for maintaining trust.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness

Case Study 5: Integrity Verification in Cloud Data Storage

Scenario

A cloud provider uses hashing to verify file integrity, but users report that files have been modified. Investigation reveals that the hash-validation mechanism was implemented incorrectly.

Analysis

Hashing can detect changes only when the trusted reference hash is protected from unauthorized modification.

For example:

Original File

↓

SHA-256

↓

Hash A

If an attacker modifies both the file and the stored hash:

Modified File

↓

SHA-256

↓

Hash B

The system may compare the modified file against the attacker's modified hash and incorrectly conclude that the file is valid.

Security Impact

- Unauthorized modifications may remain undetected.
- Users may receive corrupted files.
- Important cloud data may lose integrity.
- Attackers may manipulate stored reference hashes.
- Simple hashing alone does not provide authentication.

Recommended Solution

Use HMAC-SHA-256 when the cloud service and verification system share a secret key.

For stronger independent verification, use a digital signature:

File

↓

SHA-256

↓

Digital Signature

↓

Cloud Storage

The user's public key can then verify whether the file originated from the trusted source and whether it has been modified.

Justification

A simple hash provides change detection but does not authenticate the source. HMAC adds a secret key, while digital signatures provide integrity, authentication, and support for non-repudiation.

Conclusion

The cloud system failed because the hash-validation process did not adequately protect the reference value. HMAC or digital signatures with secure key management provide a much stronger integrity-verification mechanism.

Overall Analytical Comparison

Case Study	Cryptographic/Authentication Technique	Main Problem	Recommended Solution
1	Hashing	Unsalted SHA-1	Argon2id/bcrypt/PBKDF2 + salt
2	Digital Signature	Private-key compromise	HSM + MFA + revocation
3	Kerberos	Clock synchronization	Secure NTP + monitoring
4	X.509 / PKI	CA compromise	CT + CRL/OCSP + CA protection

Case Study	Cryptographic/Authentication Technique	Main Problem	Recommended Solution
5	Hashing	Improper validation	HMAC-SHA-256 / Digital Signature